



Powered by
Arizona State University®

Univerzitet Donja Gorica

Fakultet za Informacione Sisteme i Tehnologije

Domaći zadatak 2 : TCP i UDP komunikacija u Javi

Kompjuterske mreže

Predavači:

Prof. Dr Tomo Popović

Prof. Dr Nenad Krajnović

Student :

Irena Selčanin

23/004

1.UVOD	3
2. TCP KOMUNIKACIJA	3
2.1. IMPLEMENTACIJA TCP SERVERA	3
2.2 IMPLEMENTACIJA TCP KLIJENTA I RAZMJENA PORUKA	4
3. UDP KOMUNIKACIJA	4
3.1 IMPLEMENTACIJA UDP SERVERA	5
3.2 IMPLEMENTACIJA UDP KLIJENTA	6
4. EKSPERIMENT SA 100 PORUKA	6
4.1 TCP EKSPERIMENT – RAZMJENA 100 PORUKA	7
4.2 UDP EKSPERIMENT – RAZMJENA 100 PORUKA	8
5. POREĐENJE TCP I UDP PROTOKOLA	9
6. ANALIZA KOMUNIKACIJE POMOĆU WIRESHARK ALATA	11
6.1 ANALIZA TCP KOMUNIKACIJE	11
7. AI DNEVNIK	14
8.ZAKLJUČAK	14

1.Uvod

Cilj ovog domaćeg zadatka bio je implementirati osnovnu TCP i UDP komunikaciju između dvije Java aplikacije, kao i analizirati razlike između ova dva protokola pomoću Wireshark alata.

U radu su implementirani:

- TCP server i TCP klijent
- UDP server i UDP klijent
- razmjena 5 poruka
- eksperiment sa 100 poruka

Pored implementacije, izvršena je analiza mrežnog saobraćaja korišćenjem Wiresharka.

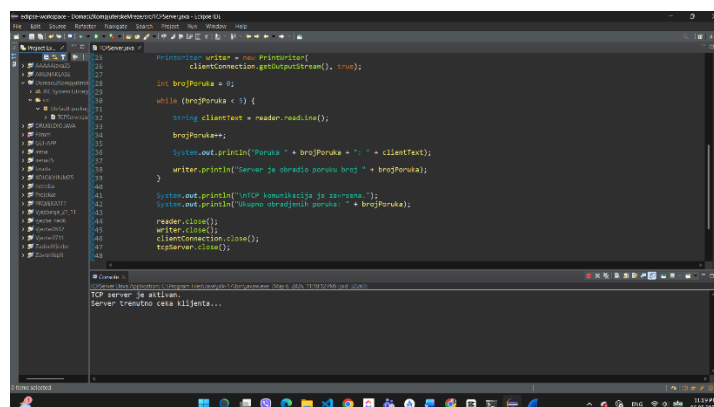
2. TCP komunikacija

TCP komunikacija realizovana je između dvije Java aplikacije korišćenjem klijent-server modela. Za implementaciju su korišćene klase `ServerSocket` i `Socket`, koje omogućavaju uspostavljanje pouzdane mrežne komunikacije između aplikacija.

2.1. Implementacija TCP servera

Za realizaciju TCP komunikacije korišćene su Java klase `ServerSocket` i `Socket`. TCP server je implementiran tako da osluškuje dolazne konekcije na unaprijed definisanom portu.

Za TCP komunikaciju korišćen je port 5055 koji je definisan u server i klijent aplikaciji kako bi se omogućila međusobna komunikacija. Nakon pokretanja server aplikacije server prelazi u stanje čekanja dok se klijent ne poveže. Kada klijent uspostavi konekciju, server prihvata komunikaciju i omogućava razmjenu poruka između aplikacija.



```
1 import java.io.*;
2 import java.net.*;
3
4 public class TCPServer {
5     public static void main(String[] args) {
6         try {
7             ServerSocket serverSocket = new ServerSocket(5055);
8             System.out.println("TCP server je pokrenut.");
9             while (true) {
10                 Socket clientSocket = serverSocket.accept();
11                 BufferedReader reader = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));
12                 String clientText = reader.readLine();
13                 System.out.println("Poruka: " + clientText);
14                 writer.println("Server je primio poruku: " + clientText);
15                 reader.close();
16                 clientSocket.close();
17             }
18         } catch (IOException e) {
19             e.printStackTrace();
20         }
21     }
22 }
```

Console Output:

```
TCP server je pokrenut.
Server trenutno čeka klijenta...
```

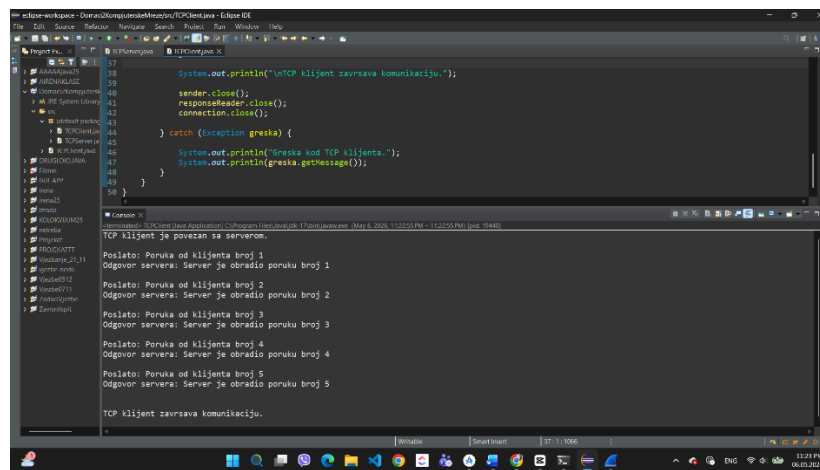
Implementacija TCP servera i pokretanje server aplikacije

2.2 Implementacija TCP klijenta i razmjena poruka

TCP klijent je povezan sa serverom korišćenjem lokalne adrese localhost i porta 5055. Nakon uspostavljanja konekcije klijent šalje tekstualne poruke serveru, dok server za svaku primljenu poruku vraća odgovor kao potvrdu uspjehnog prijema.

U okviru osnovne implementacije realizovana je razmjena ukupno 5 poruka između TCP klijenta i TCP servera.

Na osnovu prikazanog rezultata može se vidjeti da su sve poruke uspješno poslate i primljene, bez gubitka podataka i uz očuvan redosljed komunikacije, što predstavlja jednu od glavnih karakteristika TCP protokola.



```
17      System.out.println("TCP klijent završava komunikaciju.");
18
19      sender.close();
20      responseReader.close();
21      connection.close();
22
23      } catch (Exception greska) {
24
25      }
26
27      System.out.println("Greska kod TCP klijenta.");
28      System.out.println(greska.getMessage());
29
30      }
31
32      }
33
34      }
35
36      }
37
38      }
39
40      }
41
42      }
43
44      }
45
46      }
47
48      }
49
50      }
51
52      }
53
54      }
55
56      }
57
58      }
59
60      }
61
62      }
63
64      }
65
66      }
67
68      }
69
70      }
71
72      }
73
74      }
75
76      }
77
78      }
79
80      }
81
82      }
83
84      }
85
86      }
87
88      }
89
90      }
91
92      }
93
94      }
95
96      }
97
98      }
99
100     }
```

Console Output:

```
TCP klijent je povezan sa serverom.
Poslato: Poruka od klijenta broj 1
Odgovor servera: Server je obradio poruku broj 1
Poslato: Poruka od klijenta broj 2
Odgovor servera: Server je obradio poruku broj 2
Poslato: Poruka od klijenta broj 3
Odgovor servera: Server je obradio poruku broj 3
Poslato: Poruka od klijenta broj 4
Odgovor servera: Server je obradio poruku broj 4
Poslato: Poruka od klijenta broj 5
Odgovor servera: Server je obradio poruku broj 5
TCP klijent završava komunikaciju.
```

TCP klijent i razmjena poruka sa serverom

Tokom komunikacije klijent i server razmjenjuju podatke sekvencijalno, odnosno poruke se obrađuju redom kojim su poslate. Nakon završetka razmjene poruka konekcija se zatvara oslobađanjem korišćenih resursa.

Na prikazanom rezultatu moguće je vidjeti ispis poslatih poruka od strane klijenta, kao i odgovore servera koji potvrđuju uspješnu obradu svake poruke.

3. UDP komunikacija

UDP komunikacija realizovana je između dvije Java aplikacije korišćenjem DatagramSocket i DatagramPacket klasa. Za razliku od TCP protokola, UDP ne uspostavlja konekciju između klijenta i servera prije slanja podataka, već se poruke šalju direktno u obliku datagrama.

3.1 Implementacija UDP servera

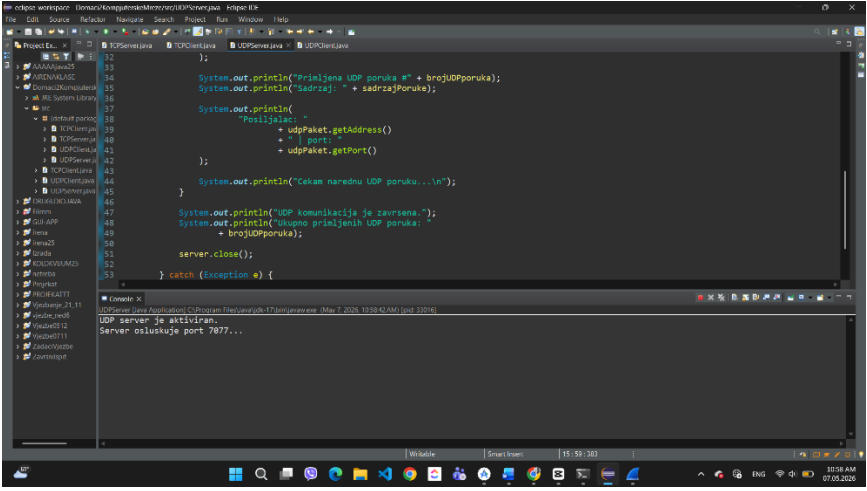
UDP server implementiran je korišćenjem Java klasa DatagramSocket i DatagramPacket. Za razliku od TCP komunikacije, kod UDP protokola ne postoji prethodno uspostavljanje konekcije između aplikacija, već server direktno prima UDP pakete koji stižu preko definisanog porta.

U programu je korišćen port 7077, preko kojeg server osluškuje dolazne UDP poruke od klijenta. Tokom testiranja korišćena je lokalna adresa localhost, jer su klijent i server pokretani na istom računaru.

Nakon pokretanja aplikacije server prelazi u stanje osluškivanja i čeka prijem UDP paketa. Kada paket stigne, server očitava sadržaj poruke, prikazuje tekst primljene poruke, kao i informacije o pošiljaocu i portu sa kojeg je poruka poslata.

Za prijem poruka korišćen je objekat klase DatagramPacket, koji omogućava čuvanje sadržaja UDP paketa zajedno sa informacijama o mrežnoj adresi i portu pošiljaoca.

UDP komunikacija omogućava bržu razmjenu podataka zbog manjeg broja kontrolnih mehanizama, ali ne garantuje potvrdu prijema niti redosljed poruka. Ipak, u ovom lokalnom testiranju sve UDP poruke uspješno su primljene i obrađene bez greške.



```
1 import java.net.*;
2 import java.util.*;
3
4 public class UDPServer {
5     public static void main(String[] args) {
6         try {
7             DatagramSocket ds = new DatagramSocket(7077);
8             System.out.println("Primljena UDP poruka");
9             System.out.println("Adresa: " + ds.getAddress());
10            System.out.println("Port: " + ds.getPort());
11            System.out.println("Posiljalac: " + ds.getAddress());
12            System.out.println("Port: " + ds.getPort());
13            System.out.println("Cekam narednu UDP poruku...");
14            while (true) {
15                DatagramPacket dp = new DatagramPacket(new byte[1024], 1024, ds);
16                ds.receive(dp);
17                System.out.println("UDP komunikacija je završena.");
18                System.out.println("Ukupno primljenih UDP poruka: " + brojUDPporuka);
19                server.close();
20            } catch (Exception e) {
21                e.printStackTrace();
22            }
23        } catch (Exception e) {
24            e.printStackTrace();
25        }
26    }
27 }
```

Console Output:

```
UDP server je aktiviran.
Server osluškuje port 7077...
```

Pokretanje UDP servera i osluškivanje porta 7077

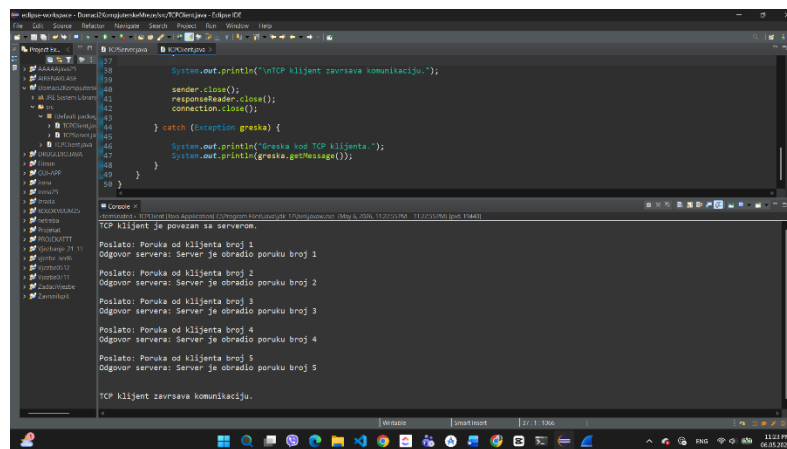
Na prikazanoj slici može se vidjeti implementacija UDP server aplikacije u Eclipse razvojnom okruženju. U konzoli je prikazano da je UDP server uspješno pokrenut i da osluškuje port 7077, preko kojeg očekuje dolazne UDP poruke od klijenta.

3.2 Implementacija UDP klijenta

UDP klijent implementiran je korišćenjem klasa DatagramSocket i DatagramPacket. Klijent šalje poruke serveru preko definisanog porta bez prethodnog uspostavljanja konekcije, što predstavlja osnovnu karakteristiku UDP protokola.

U programu je korišćena lokalna adresa localhost, jer se komunikacija izvršava na istom računaru. Za UDP komunikaciju korišćen je port **7077**, koji je definisan i u server i u klijent aplikaciji.

Tokom rada klijent formira UDP poruke, pretvara ih u niz bajtova i šalje serveru pomoću UDP paketa.



```
27      System.out.println("UDP klijent završava komunikaciju.");
28
29      sender.close();
30      responder.close();
31      connection.close();
32
33      } catch (Exception greska) {
34
35      }
36
37      System.out.println("Greska kod TCP klijenta.");
38      System.out.println(greska.getMessage());
39  }
40  }
41
42  }
43
44  }
45
46  }
47
48  }
49  }
50  }
```

Console Output:

```
TCP klijent je povezan sa serverom.
Poslato: Poruka od klijenta broj 1
Odgovor servera: Server je obradio poruku broj 1
Poslato: Poruka od klijenta broj 2
Odgovor servera: Server je obradio poruku broj 2
Poslato: Poruka od klijenta broj 3
Odgovor servera: Server je obradio poruku broj 3
Poslato: Poruka od klijenta broj 4
Odgovor servera: Server je obradio poruku broj 4
Poslato: Poruka od klijenta broj 5
Odgovor servera: Server je obradio poruku broj 5
TCP klijent završava komunikaciju.
```

Razmjena UDP poruka između klijenta i servera

Na prikazanom rezultatu može se vidjeti uspješno slanje ukupno 5 UDP poruka prema serveru. Za svaku poslatu poruku prikazan je tekst poruke koji je poslat preko mreže.

Za razliku od TCP komunikacije, kod UDP protokola ne postoji potvrda prijema poruke niti uspostavljanje stalne konekcije između aplikacija. Ipak, u ovom primjeru sve poruke su uspješno poslate i primljene bez greške.

Nakon završetka razmjene poruka UDP klijent zatvara komunikaciju i oslobađa korišćene resurse.

4. Eksperiment sa 100 poruka

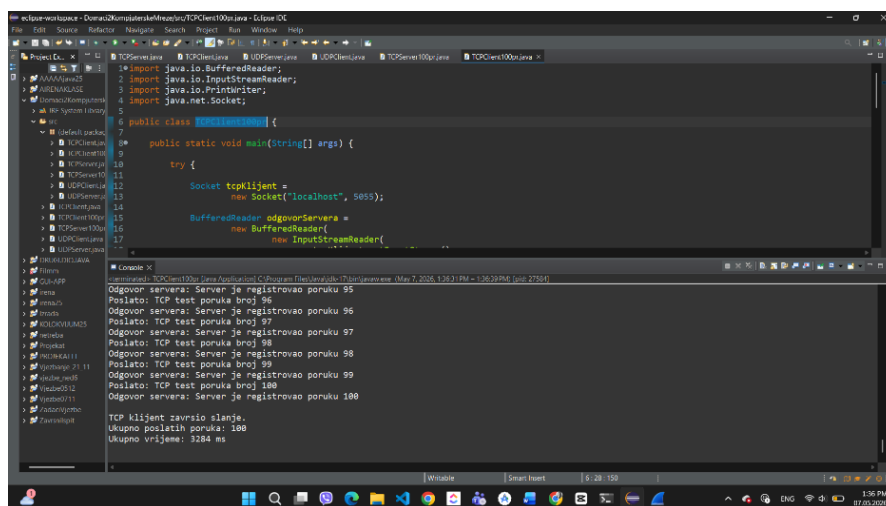
Nakon realizacije osnovne TCP i UDP komunikacije izvršen je dodatni eksperiment u kojem je između klijenta i servera razmijenjeno ukupno 100 poruka.

Cilj eksperimenta bio je provjera funkcionisanja komunikacije tokom većeg broja razmjena podataka, kao i evidentiranje ukupnog vremena potrebnog za slanje i prijem poruka kod TCP i UDP protokola.

Tokom izvođenja eksperimenta aplikacije su evidentirale broj poslatih i primljenih poruka, kao i ukupno vrijeme trajanja komunikacije.

4.1 TCP eksperiment – razmjena 100 poruka

U okviru TCP eksperimenta realizovana je razmjena ukupno 100 poruka između TCP klijenta i TCP servera. Cilj eksperimenta bio je provjera stabilnosti komunikacije tokom većeg broja razmjena podataka, kao i mjerenje ukupnog vremena potrebnog za izvršavanje komunikacije. TCP klijent je slao poruke serveru sekvencijalno, dok je server za svaku primljenu poruku vraćao odgovor kao potvrdu uspješnog prijema. Tokom komunikacije evidentiran je ukupan broj obrađenih poruka i vrijeme trajanja komunikacije. Tokom slanja poruka korišćeno je kratko vremensko kašnjenje između poruka kako bi komunikacija bila preglednija i lakša za analizu u konzoli i Wireshark alatu.



```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.Socket;

public class TCPClient100p {

    public static void main(String[] args) {
        try {
            Socket tcpKlijent =
                new Socket("localhost", 9855);

            BufferedReader odgovorServera =
                new BufferedReader(
                    new InputStreamReader(
                        tcpKlijent.getInputStream()));

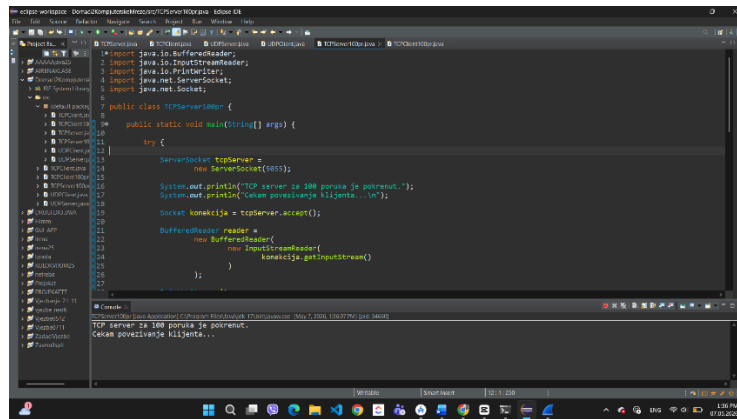
            // ... (rest of the code) ...

            TCP klijent završio slanje.
            Ukupno poslanih poruka: 100
            Ukupno vrijeme: 3284 ms
```

TCP eksperiment – razmjena 100 poruka između klijenta i servera

Na osnovu rezultata može se vidjeti da je svih 100 poruka uspješno poslato i primljeno bez gubitka podataka. Takođe je provjeren redosljed poruka, pri čemu je potvrđeno da TCP protokol isporučuje poruke istim redoslijedom kojim su poslate.

TCP protokol obezbjeđuje pouzdanu komunikaciju zahvaljujući kontroli konekcije i potvrdi prijema podataka, ali zbog dodatnih kontrolnih mehanizama komunikacija traje nešto duže u odnosu na UDP protokol.



```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.io.OutputStreamWriter;
import java.net.ServerSocket;
import java.net.Socket;

public class TCPServer {
    public static void main(String[] args) {
        try {
            ServerSocket tcpServer =
                new ServerSocket(8080);
            System.out.println("TCP server is 8080 port is started.");
            System.out.println("Cekanje povezivanja klijenta...");
            Socket konekcija = tcpServer.accept();
            BufferedReader reader =
                new BufferedReader(
                    new InputStreamReader(
                        konekcija.getInputStream()
                    )
                );
            // ... (rest of the code) ...
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

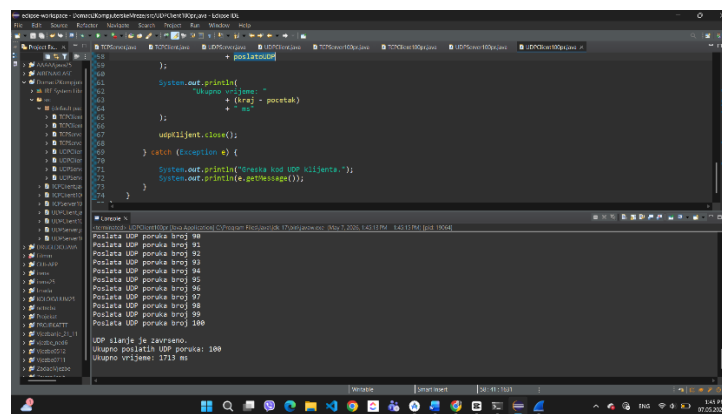
TCP server – prijem 100 poruka

Na server strani evidentiran je prijem svih poruka koje su uspješno obrađene tokom komunikacije. Nakon završetka eksperimenta konekcija je zatvorena i oslobođeni su korišćeni mrežni resursi.

4.2 UDP eksperiment – razmjena 100 poruka

U okviru UDP eksperimenta realizovana je razmjena ukupno 100 poruka između UDP klijenta i UDP servera. Cilj eksperimenta bio je analiza rada UDP protokola tokom većeg broja razmjena podataka i evidentiranje vremena potrebnog za komunikaciju.

UDP klijent slao je poruke serveru korišćenjem UDP paketa, bez uspostavljanja stalne konekcije između aplikacija. Tokom komunikacije evidentiran je broj poslatih poruka i ukupno vrijeme trajanja slanja podataka.



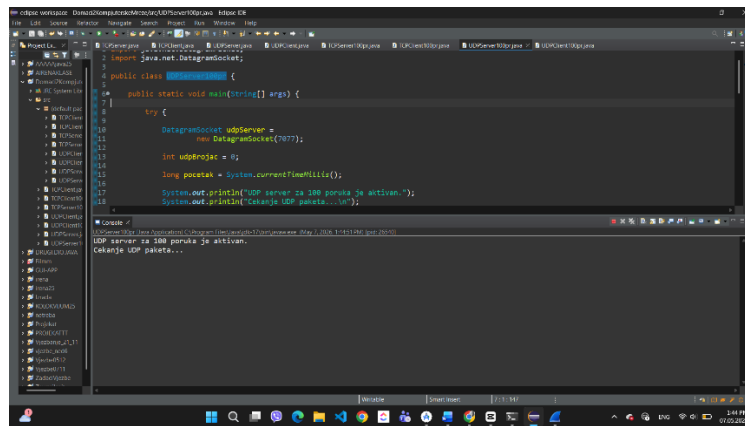
```
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.SocketException;
import java.util.Date;

public class UDPClient {
    public static void main(String[] args) {
        try {
            DatagramSocket socket =
                new DatagramSocket();
            // ... (rest of the code) ...
            System.out.println("UDP klijent je završeno.");
            System.out.println("Ukupno poslata UDP poruka: 100.");
            System.out.println("Ukupno vrijeme: 1713 ms.");
        } catch (SocketException e) {
            e.printStackTrace();
        }
    }
}
```

UDP klijent – slanje 100 UDP poruka

Na osnovu prikazanog rezultata može se vidjeti da je uspješno poslato ukupno 100 UDP poruka. UDP komunikacija izvršena je brže u odnosu na TCP komunikaciju zbog manjeg broja kontrolnih operacija i nepostojanja potvrde prijema poruka.

Za razliku od TCP protokola, UDP ne garantuje pouzdanu isporuku podataka niti očuvanje redosljeda poruka. Ipak, u ovom primjeru sve poruke su uspješno pristigle serveru.



```
import java.net.DatagramSocket;

public class UDPoverTCP {

    public static void main(String[] args) {
        try {
            DatagramSocket udpServer =
                new DatagramSocket(7877);

            int udpPaket = 0;

            long pocetak = System.currentTimeMillis();

            System.out.println("UDP server za 100 poruka je aktivan.");
            System.out.println("Cekanje UDP paketa...");

            while (udpPaket < 100) {
                byte[] data = new byte[1024];
                DatagramPacket packet = new DatagramPacket(data, data.length);
                udpServer.receive(packet);
                System.out.println("Prijem UDP paketa: " + packet.getData());
                udpPaket++;
            }

            long kraj = System.currentTimeMillis();
            System.out.println("Vrijeme trajanja: " + (kraj - pocetak) + " ms");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

UDP server – prijem 100 UDP poruka

Na server strani evidentiran je prijem UDP poruka koje su uspješno obrađene tokom eksperimenta. Nakon završetka komunikacije server zatvara socket i oslobađa korišćene resurse.

5. Poređenje TCP i UDP protokola

Na osnovu implementacije i eksperimenata može se jasno uočiti razlika između TCP i UDP protokola. Oba protokola se koriste za prenos podataka preko mreže, ali se razlikuju po načinu rada, pouzdanosti, brzini i kontroli prenosa.

TCP je konekcijski protokol, što znači da se prije slanja podataka prvo mora uspostaviti konekcija između klijenta i servera. Zbog toga se kod TCP komunikacije koristi handshake proces. Nakon uspostavljanja konekcije, poruke se šalju redom, a primalac potvrđuje njihov prijem. Ovo omogućava pouzdanu komunikaciju i smanjuje mogućnost gubitka podataka. U mom TCP eksperimentu, klijent je slao poruke serveru, a server je za svaku poruku vraćao odgovor. Na taj način se moglo vidjeti da je svaka poruka uspješno primljena i obrađena. Kod eksperimenta sa 100 poruka sve poruke su stigle redom kojim su poslate. Ovo potvrđuje da TCP čuva redosljed poruka i obezbeđuje pouzdan prenos.

UDP je jednostavniji protokol i ne uspostavlja konekciju prije slanja podataka. Kod UDP komunikacije poruke se šalju direktno u obliku paketa, odnosno datagrama. UDP ne šalje potvrdu da je poruka primljena i ne garantuje da će poruke stići istim redosljedom kojim su poslate.

U mom UDP eksperimentu poruke su takođe uspješno razmijenjene, jer je testiranje rađeno lokalno na istom računaru preko localhost adrese. U takvom okruženju je mala mogućnost gubitka paketa. Ipak, važno je naglasiti da UDP kao protokol ne garantuje pouzdanost, pa se u realnoj mreži može desiti da neki paket ne stigne ili da poruke stignu drugačijim redoslijedom. Glavna prednost TCP protokola je pouzdanost. On je pogodan za aplikacije kod kojih je važno da svi podaci stignu tačno i redom. Primjeri su slanje fajlova, web komunikacija, email i aplikacije gdje gubitak podataka nije prihvatljiv.

Glavna prednost UDP protokola je brzina. Pošto nema uspostavljanja konekcije, potvrda i dodatne kontrole, UDP može biti brži od TCP-a. Zbog toga se koristi u aplikacijama gdje je važnija brzina nego potpuna pouzdanost, kao što su video pozivi, online igre i streaming. U eksperimentu sa 100 poruka UDP komunikacija je završena brže od TCP komunikacije. Razlog je to što UDP šalje pakete bez dodatnih potvrda, dok TCP za svaku razmjenu koristi dodatne mehanizme kontrole i potvrde prijema.

TCP i UDP imaju različitu namjenu. TCP je bolji izbor kada je važna sigurnost, tačnost i redosljed podataka, dok je UDP bolji kada je važna brzina i kada aplikacija može da podnese eventualni gubitak nekog paketa.

Karakteristika	TCP	UDP
Uspostavljanje konekcije	Da, koristi handshake	Ne uspostavlja konekciju
Pouzdanost	Pouzdan protokol	Nema garanciju isporuke
Redosljed poruka	Čuva redosljed	Ne garantuje redosljed
Potvrda prijema	Postoji ACK potvrda	Nema potvrde prijema
Brzina	Sporiji zbog dodatnih kontrola	Brži i jednostavniji
Primjena	Fajlovi, web, email	Streaming, igre, video pozivi
Eksperiment	100 poruka uspješno i redom	100 poruka uspješno u lokalnom testu

6. Analiza komunikacije pomoću Wireshark alata

Za analizu mrežnog saobraćaja korišćen je alat Wireshark. Pomoću ovog alata moguće je pratiti razmjenu paketa između klijenta i servera, kao i analizirati način rada TCP i UDP protokola. Tokom analize korišćeni su odgovarajući filteri kako bi bili prikazani samo paketi vezani za realizovane komunikacije.

6.1 Analiza TCP komunikacije

Za analizu TCP komunikacije korišćen je Wireshark alat uz filter:

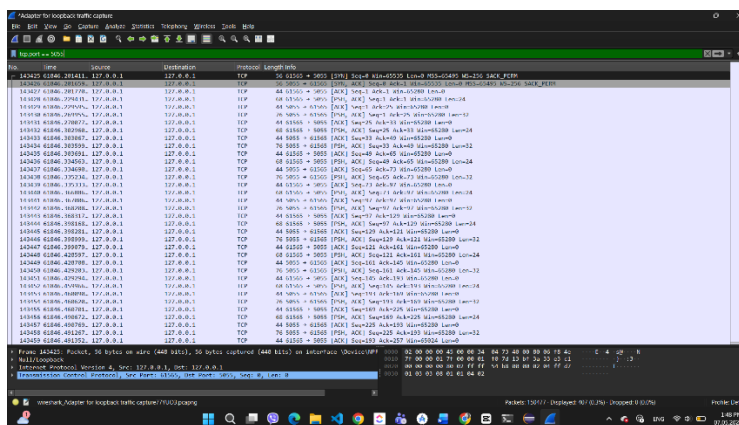
tcp.port == 5055

TCP protokol prije slanja podataka prvo uspostavlja konekciju između klijenta i servera. Taj proces naziva se *TCP three-way handshake* i predstavlja osnovni mehanizam kojim TCP omogućava pouzdanu komunikaciju.

Handshake proces sastoji se iz tri koraka:

- SYN – klijent šalje zahtjev serveru za uspostavljanje konekcije.
- SYN-ACK – server prihvata zahtjev i šalje odgovor klijentu.
- ACK – klijent potvrđuje prijem odgovora servera, nakon čega je konekcija uspješno uspostavljena.

Na osnovu prikaza u Wireshark alatu može se vidjeti pravilna razmjena ovih paketa, što potvrđuje da je TCP konekcija uspješno ostvarena između klijenta i servera korišćenjem porta 5055.



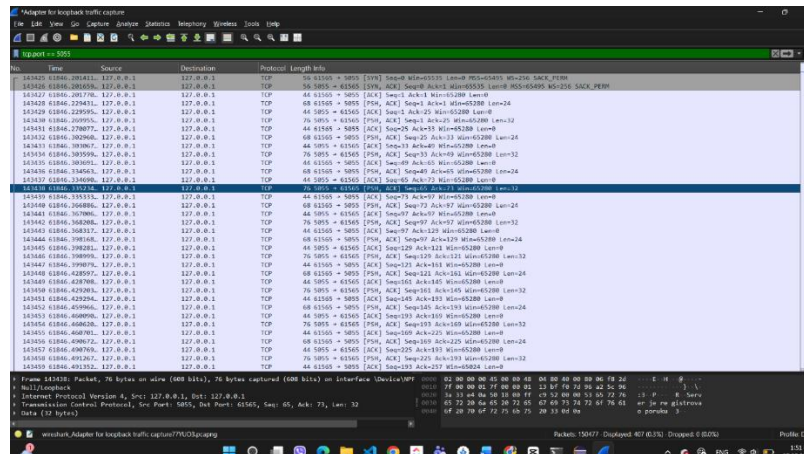
TCP three-way handshake između klijenta i servera.

Nakon uspostavljanja konekcije započinje razmjena aplikativnih poruka između TCP klijenta i servera. U Wireshark prikazu mogu se vidjeti paketi sa oznakama PSH i ACK.

- Oznaka PSH označava slanje aplikativnih podataka.
- Oznaka ACK predstavlja potvrdu prijema podataka od druge strane komunikacije.

U donjem dijelu prikaza vidljiv je sadržaj aplikativne poruke koju server šalje klijentu. Na taj način potvrđuje se da TCP protokol uspješno prenosi podatke bez gubitka poruka i uz očuvanje njihovog redosljeda.

Takođe, može se primijetiti da TCP nakon svake razmjene vrši potvrdu prijema podataka, što dodatno doprinosi sigurnosti i pouzdanosti komunikacije.



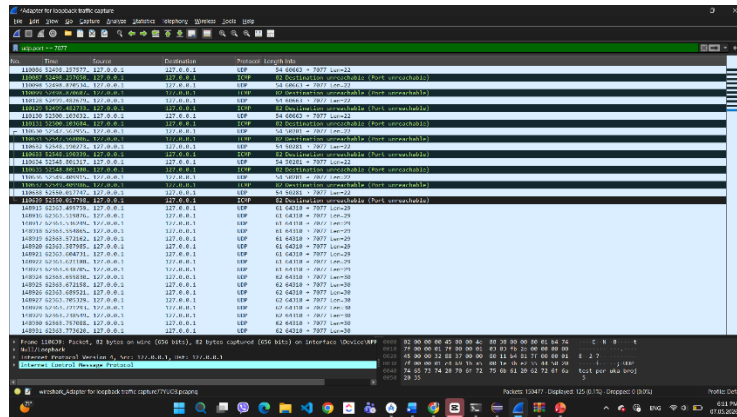
TCP aplikativna komunikacija između klijenta i servera.

6.2 Analiza UDP komunikacije u Wireshark-u

Za analizu UDP komunikacije korišćen je Wireshark alat uz filter `udp.port == 7077`, kako bi bili prikazani samo UDP paketi koji pripadaju realizovanoj komunikaciji između klijenta i servera.

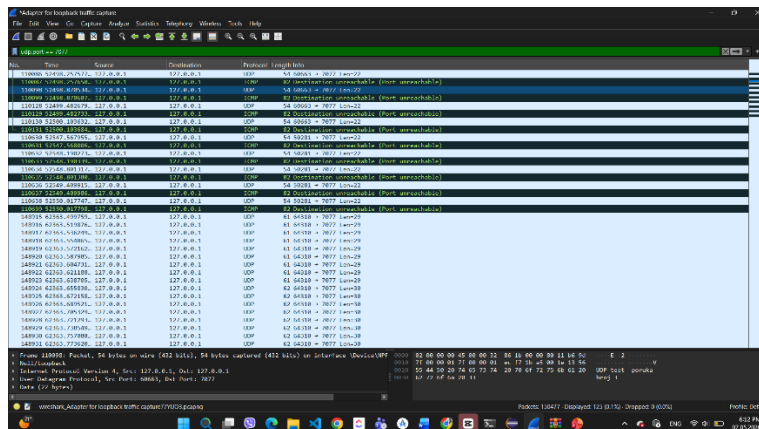
Tokom analize posmatrani su izvorna i odredišna adresa, broj portova, veličina paketa, kao i sadržaj same poruke.

Nakon pokretanja UDP klijenta i servera, Wireshark je registrovao UDP pakete koji su slati preko porta 7077. Za razliku od TCP protokola, kod UDP komunikacije ne postoji uspostavljanje konekcije niti handshake proces, već se poruke šalju direktno bez potvrde prijema.



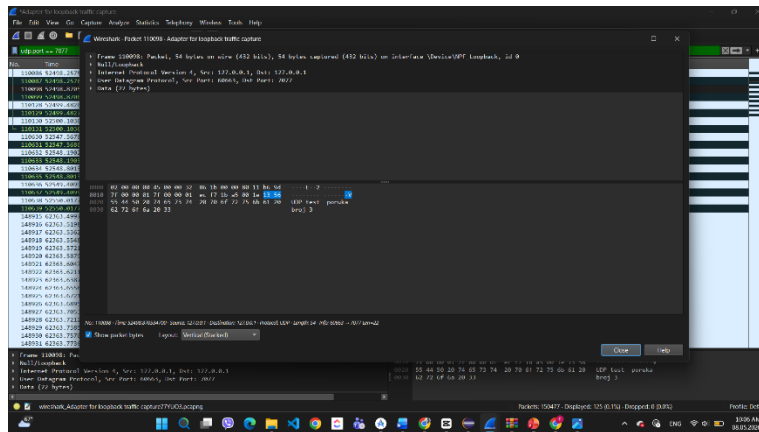
Prikaz UDP paketa u Wireshark-u

Na prikazanom screenshotu mogu se vidjeti UDP paketi koji se šalju između lokalnih adresa 127.0.0.1 koristeći port 7077. U koloni *Protocol* prikazan je UDP protokol, dok kolona *Info* sadrži informacije o izvornom i odredišnom portu, kao i veličini paketa. Pored UDP paketa mogu se primijetiti i ICMP poruke *Destination unreachable*, koje se mogu pojaviti kada server u određenom trenutku nije aktivan ili ne odgovara na poslani paket.



Detaljan prikaz UDP paketa i portova

Detaljnijim pregledom jednog UDP paketa mogu se vidjeti osnovni podaci o komunikaciji. Prikazana je izvorna IP adresa 127.0.0.1, odredišna IP adresa 127.0.0.1, kao i izvorni port 60663 i odredišni port 7077. Takođe je prikazana veličina UDP paketa i njegov sadržaj.



Prikaz sadržaja UDP poruke u Wireshark-u

U okviru sadržaja paketa prikazana je tekstualna poruka „*UDP test poruka broj 3*“, što potvrđuje da je aplikativni sadržaj uspješno prenesen putem UDP protokola. Na osnovu prikazane komunikacije može se zaključiti da UDP omogućava jednostavan i brz prenos podataka, ali bez garancije isporuke i bez provjere redosljedja paketa.

7. AI dnevnik

Tokom izrade domaćeg zadatka korišćen je AI alat ChatGPT kao dodatna pomoć pri razumijevanju TCP i UDP socket komunikacije, organizaciji pojedinih djelova koda i pisanju izvještaja.

AI alat je korišćen za pomoć pri razumijevanju Java socket komunikacije, dodatno objašnjenje razlika između TCP i UDP protokola, organizaciju strukture izvještaja, pomoć pri analizi mrežnog saobraćaja u Wireshark alatu...

Implementacija aplikacija, testiranje komunikacije i provjera rezultata izvršeni su kroz praktičan rad u Java programskom jeziku korišćenjem Eclipse razvojnog okruženja.

8. Zaključak

Kroz realizaciju ovog domaćeg zadatka uspješno su implementirane osnovne TCP i UDP komunikacije između Java aplikacija korišćenjem socket programiranja. Tokom rada razvijeni su TCP server i klijent, kao i UDP server i klijent, pri čemu je izvršena razmjena poruka između aplikacija i analiziran način rada oba protokola.

Prvi dio zadatka odnosio se na osnovnu razmjenu 5 poruka između klijenta i servera. Na osnovu rezultata moglo se vidjeti da TCP komunikacija funkcioniše pouzdano, uz očuvanje redosljedja

poruka i potvrdu prijema podataka. Kod UDP komunikacije poruke su takođe uspješno razmijenjene, ali bez uspostavljanja konekcije i bez dodatnih mehanizama potvrde prijema.

Nakon osnovne implementacije izvršen je eksperiment sa razmjenom 100 poruka za TCP i UDP protokol. Tokom eksperimenta evidentiran je broj poslatih i primljenih poruka, kao i ponašanje komunikacije tokom većeg broja razmjena podataka. Na osnovu dobijenih rezultata moglo se zaključiti da TCP protokol omogućava stabilnu i pouzdanu komunikaciju čak i pri većem broju poruka, dok UDP omogućava jednostavniji i brži prenos podataka zbog manjeg broja kontrolnih operacija.

Poseban dio zadatka odnosio se na analizu mrežnog saobraćaja pomoću Wireshark alata. Korišćenjem Wiresharka detaljno je analiziran TCP handshake proces, razmjena aplikativnih TCP poruka, kao i UDP paketi koji su sadržali poslate poruke. Tokom analize bilo je moguće vidjeti razliku između ova dva protokola na nivou mrežnih paketa, uključujući način uspostavljanja konekcije, potvrde prijema podataka, korišćenje portova i sadržaj aplikativnih poruka.

Kroz praktičan rad stečeno je bolje razumijevanje socket programiranja u Java programskom jeziku, rada klijent-server aplikacija i načina komunikacije preko mreže. Takođe je stečeno iskustvo u korišćenju Wireshark alata za analizu mrežnog saobraćaja i praćenje komunikacije između aplikacija.

Na osnovu realizovanih eksperimenata može se zaključiti da TCP i UDP protokoli imaju različite namjene i karakteristike. TCP je pogodniji za situacije gdje su važni pouzdanost, tačnost i očuvanje redosljeda podataka, dok je UDP pogodniji za aplikacije kod kojih je važnija brzina prenosa i manji broj kontrolnih mehanizama.

Realizacijom ovog zadatka uspješno su povezani teorijski koncepti računarskih mreža sa praktičnom implementacijom komunikacije između aplikacija, što je omogućilo jasnije razumijevanje rada mrežnih protokola i njihove primjene u stvarnim sistemima